

Lab 1

Ubuntu Installation Guide using Oracle VirtualBox

Important

For this laboratory, **Oracle VirtualBox** will be used.
macOS users may optionally use **UTM Virtual Machine**, but all demonstrations will be performed using VirtualBox.

Software Downloads

1. Download **Oracle VirtualBox** <https://www.virtualbox.org/wiki/Downloads>
2. (Optional for macOS) Download **UTM Virtual Machine** <https://mac.getutm.app/>
3. Download the latest **Ubuntu Desktop ISO (Ubuntu 26.04 LTS)** <https://ubuntu.com/download/desktop>

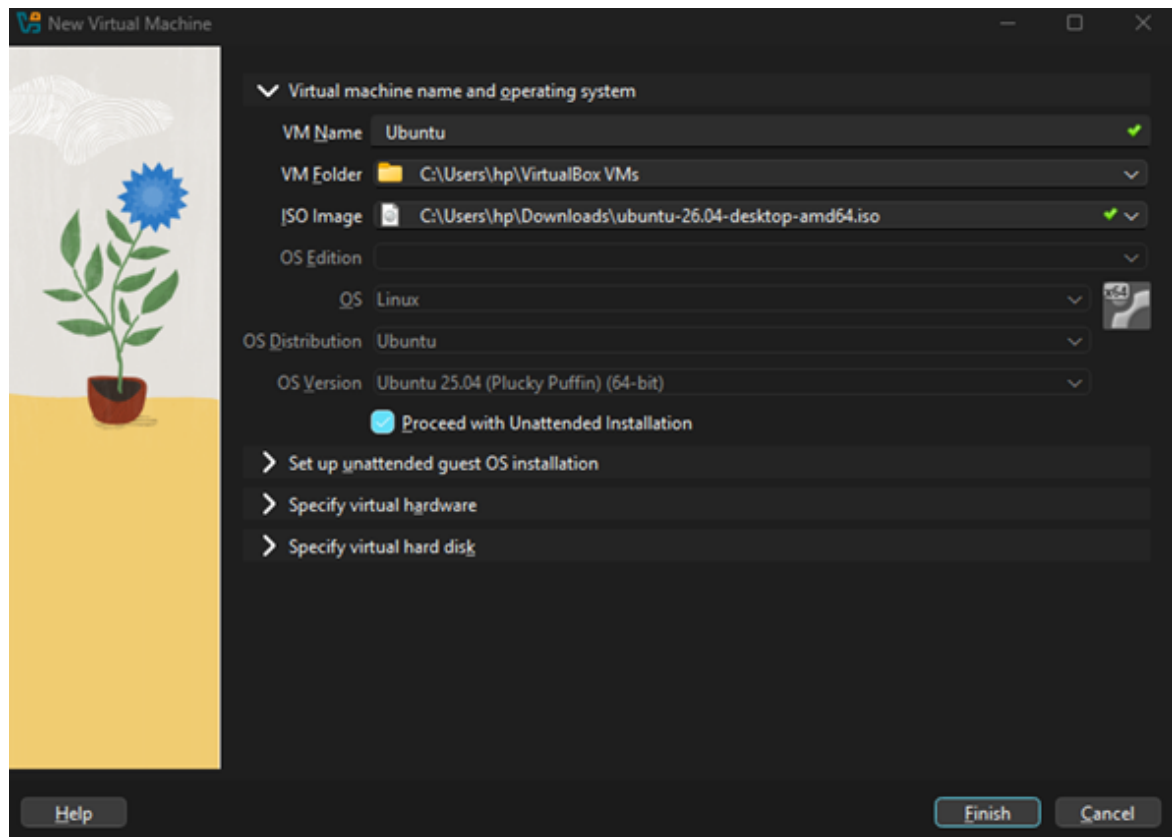
Installation Steps

1. Install Oracle VirtualBox.
2. Open VirtualBox and click **New**.
3. Enter the virtual machine details:
 - Name
 - Ubuntu (64-bit)
 - Select the downloaded Ubuntu ISO
4. Allocate at least **4096 MB RAM** (or more depending on your laptop configuration).
5. Under Boot Order:
 - Remove (Uncheck) **Floppy**
 - Keep only:
 - Optical
 - Hard Disk
6. Go to **Settings** → **System** → **Processor**. If your processor has 6 or more cores, assigning **2 CPUs or more** is recommended (Optional).
7. Start the Virtual Machine.
8. Select **Install Ubuntu** using the **Normal Installation** option.
9. During Disk Setup choose **Erase disk and install Ubuntu**.
10. Continue following the on-screen instructions to complete the installation.

Important Note

Selecting **"Erase disk and install Ubuntu"** only erases the **virtual hard disk created inside VirtualBox**.
Your Windows installation and personal files will NOT be affected.

Screenshot Placeholders



Optimise your computer



Install recommended proprietary software?

Ubuntu ships with no proprietary software by default. Installing additional software may improve your computer's performance.

- ☒ Install third-party software for graphics and Wi-Fi hardware
Including but not limited to NVIDIA drivers and similar
- ☒ Download and install support for additional media formats
Including but not limited to MP3, MP4, MOV and similar

Back



Next

Disk setup ×



How do you want to install Ubuntu?

☒ **Erase disk and install Ubuntu**
Warning: All data and partitions on the disk will be erased, including operating systems.

☐ **Manual installation**
For advanced users seeking customized disk setups.

Back

.....●.....

Next

Create your account ×

Create your account ×



Create your account

Your name

Dhairya ✓

Your computer's name

dhairya-VirtualBox ✓

Your username

dhairya ✓

Password

dhairya123 Hide Fair password

Confirm password

dhairya123 ✓

☒ Require my password to log in

☐ Use Active Directory

Back

.....●.....

Next



☐ You will need to enter a passphrase every time you turn on your computer. This uses LVM with LUKS encryption.

> Advanced options

4

Mini Demonstration

Linux Programming Laboratory

Objective

The objective of this demonstration is to familiarize students with basic Linux commands by creating, editing, copying, renaming, and deleting files and directories using the Ubuntu terminal.

Procedure

Follow the commands in the given order. Observe the output after each command.

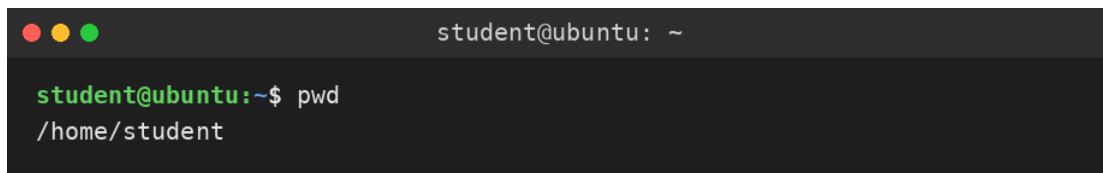
Step-by-Step Demonstration

Step 1: Check Current Working Directory

```
pwd
```

Expected Output

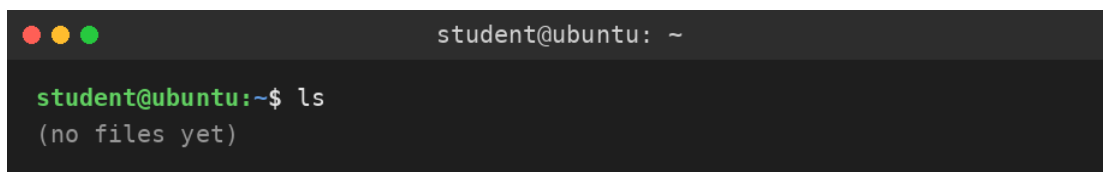
```
/home/student
```



```
student@ubuntu: ~  
  
student@ubuntu:~$ pwd  
/home/student
```

Step 2: List Files and Directories

```
ls
```



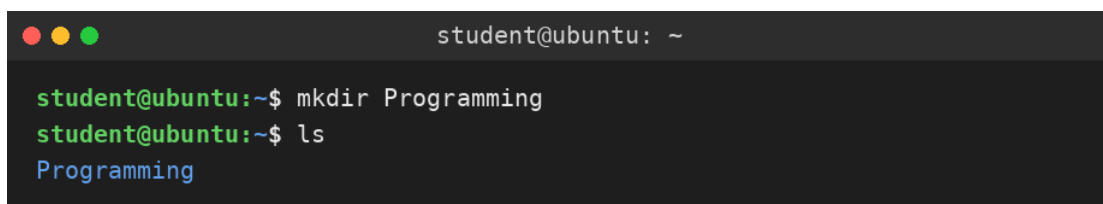
```
student@ubuntu: ~  
  
student@ubuntu:~$ ls  
(no files yet)
```

Step 3: Create a New Directory

```
mkdir Programming
```

Verify the directory.

```
ls
```



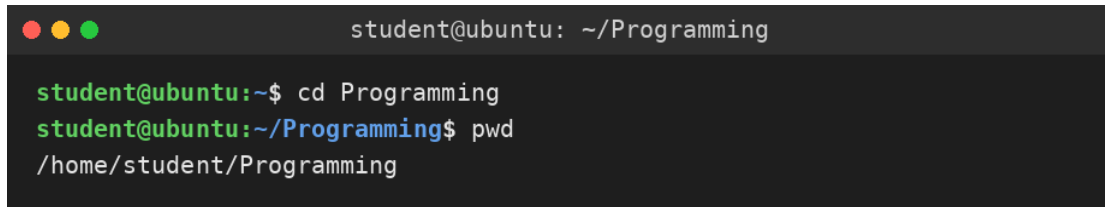
```
student@ubuntu: ~  
  
student@ubuntu:~$ mkdir Programming  
student@ubuntu:~$ ls  
Programming
```

Step 4: Move into the Directory

```
cd Programming  
pwd
```

Expected Output

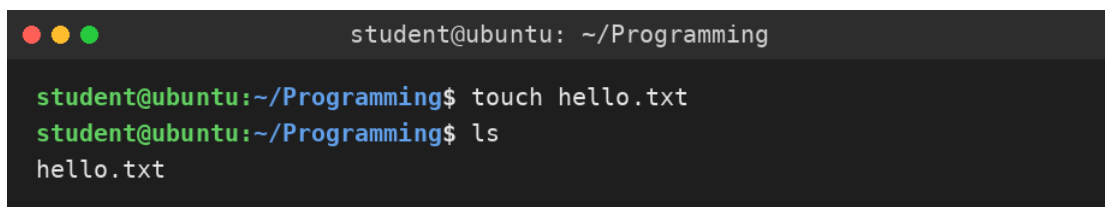
```
/home/student/Programming
```

A terminal window with a dark background and light text. The title bar shows three colored circles (red, yellow, green) and the text 'student@ubuntu: ~/Programming'. The terminal content shows the following commands and output:

```
student@ubuntu:~$ cd Programming  
student@ubuntu:~/Programming$ pwd  
/home/student/Programming
```

Step 5: Create a New File

```
touch hello.txt  
ls
```

A terminal window with a dark background and light text. The title bar shows three colored circles (red, yellow, green) and the text 'student@ubuntu: ~/Programming'. The terminal content shows the following commands and output:

```
student@ubuntu:~/Programming$ touch hello.txt  
student@ubuntu:~/Programming$ ls  
hello.txt
```

Step 6: Edit the File Using Nano

```
nano hello.txt
```

Type the following text.

```
Hello Linux  
This is my first file.
```

Save the file using:

- Ctrl + O
- Press Enter
- Ctrl + X

A screenshot of a terminal window showing the nano text editor. The title bar indicates the user is 'student@ubuntu' in the directory '~/Programming'. The editor's status bar shows 'GNU nano 6.2' and the filename 'hello.txt'. The main editing area contains two lines of text: 'Hello Linux' and 'This is my first file.'. At the bottom, a help menu lists various keyboard shortcuts: ^G Help, ^O Write Out, ^W Where Is, ^K Cut Text, ^J Justify, ^X Exit, ^R Read File, ^\ Replace, ^U Paste Text, and ^T To Spell.

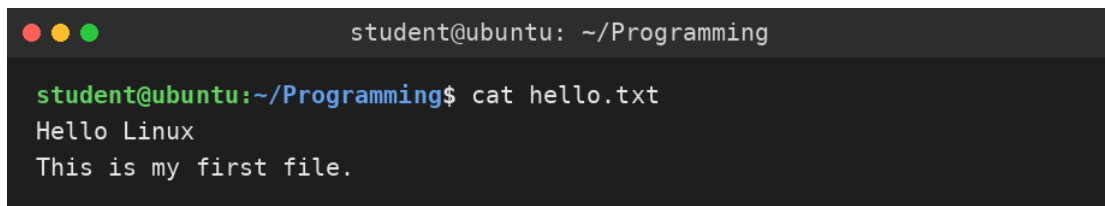
```
student@ubuntu: ~/Programming
GNU nano 6.2 hello.txt
Hello Linux
This is my first file.
^G Help    ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify
^X Exit    ^R Read File  ^\ Replace   ^U Paste Text ^T To Spell
```

Step 7: Display File Contents

```
cat hello.txt
```

Expected Output

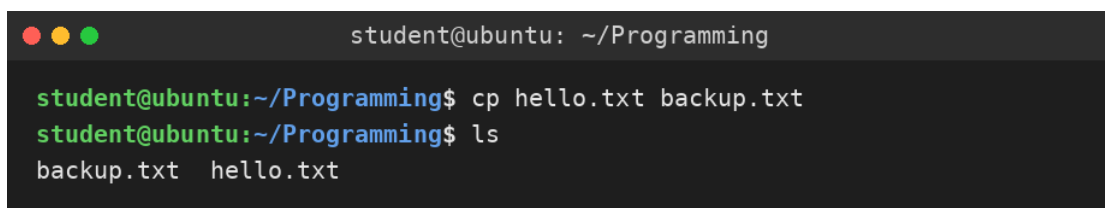
```
Hello Linux
This is my first file.
```

A screenshot of a terminal window showing the command 'cat hello.txt' being executed. The output of the command is displayed on the following lines: 'Hello Linux' and 'This is my first file.'. The terminal title bar shows 'student@ubuntu: ~/Programming'.

```
student@ubuntu: ~/Programming
student@ubuntu:~/Programming$ cat hello.txt
Hello Linux
This is my first file.
```

Step 8: Copy the File

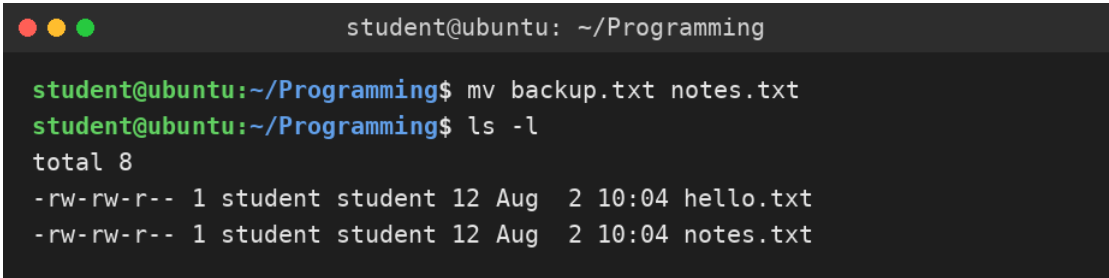
```
cp hello.txt backup.txt
ls
```

A screenshot of a terminal window showing two commands being executed: 'cp hello.txt backup.txt' and 'ls'. The output of the 'ls' command is 'backup.txt' and 'hello.txt'. The terminal title bar shows 'student@ubuntu: ~/Programming'.

```
student@ubuntu: ~/Programming
student@ubuntu:~/Programming$ cp hello.txt backup.txt
student@ubuntu:~/Programming$ ls
backup.txt  hello.txt
```

Step 9: Rename the Copied File

```
mv backup.txt notes.txt
ls -l
```

A terminal window titled 'student@ubuntu: ~/Programming' shows the following commands and output:

```
student@ubuntu:~/Programming$ mv backup.txt notes.txt
student@ubuntu:~/Programming$ ls -l
total 8
-rw-rw-r-- 1 student student 12 Aug  2 10:04 hello.txt
-rw-rw-r-- 1 student student 12 Aug  2 10:04 notes.txt
```

Step 10: Delete Files

Delete the renamed file.

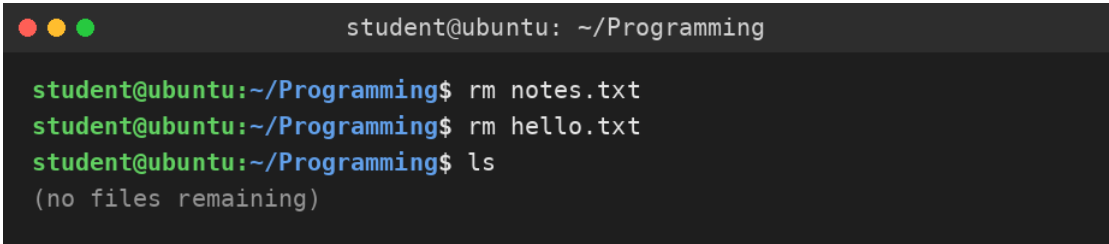
```
rm notes.txt
```

Delete the original file.

```
rm hello.txt
```

Verify.

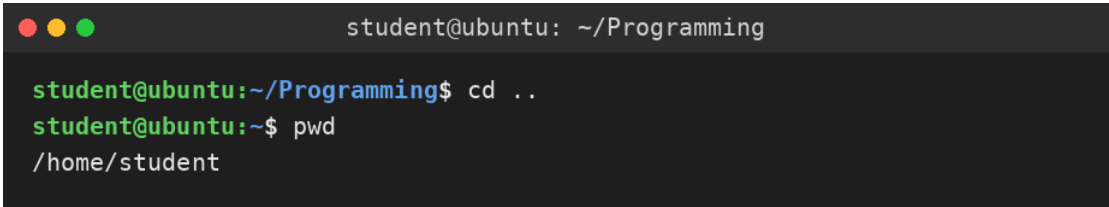
```
ls
```

A terminal window titled 'student@ubuntu: ~/Programming' shows the following commands and output:

```
student@ubuntu:~/Programming$ rm notes.txt
student@ubuntu:~/Programming$ rm hello.txt
student@ubuntu:~/Programming$ ls
(no files remaining)
```

Step 11: Return to Parent Directory

```
cd ..
pwd
```

A terminal window titled 'student@ubuntu: ~/Programming' shows the following commands and output:

```
student@ubuntu:~/Programming$ cd ..
student@ubuntu:~$ pwd
/home/student
```

Step 12: Remove the Directory

```
rmdir Programming
```

Verify.

```
ls
```

```
student@ubuntu: ~  
  
student@ubuntu:~$ rmdir Programming  
student@ubuntu:~$ ls  
(directory removed)
```

Final Directory Structure

After completing all the above steps successfully, the directory **Programming** and all files created during the demonstration should no longer exist.

C++ in Ubuntu

The following steps introduce C/C++ in the Ubuntu terminal.

Step 1: First create a cpp file using the nano editor and paste your C++ code in it:

```
nano filename.cpp
```

```
#include <iostream>  
int main()  
{  
    std::cout << "Hello LinuxHint Followers" << std::endl;  
    return 0;  
}
```

Save the file using “Ctrl+X”, add “Y” and enter to exit.

Step 2: To compile the program, navigate to the directory in which you saved the .cpp file and run the following command:

```
g++ -o output_file filename.cpp
```

Essentially, the -o option specifies the name of the output file that the compiler will create.

```
pi@raspberrypi:~ $ g++ -o output mycode.cpp  
pi@raspberrypi:~ $
```

Note: If your program includes mathematical functions:

```
g++ -o output_file filename.cpp -lm
```

Step 3: Now run the output file using the following command:

```
./output
```

This will run our code and gives you the output.

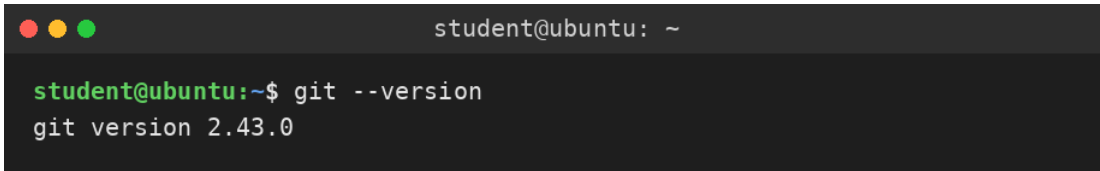
```
pi@raspberrypi:~ $ ./output  
Hello Linux Hint Followers  
pi@raspberrypi:~ $
```

GitHub Basics

The following steps introduce version control using Git and connecting a local repository to GitHub.

Step 1: Check Git Installation

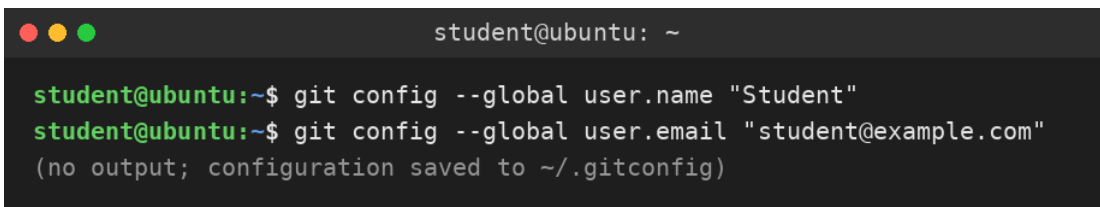
```
git --version
```

A terminal window titled 'student@ubuntu: ~' showing the command 'git --version' being executed. The output is 'git version 2.43.0'.

```
student@ubuntu:~$ git --version
git version 2.43.0
```

Step 2: Configure Git Username and Email

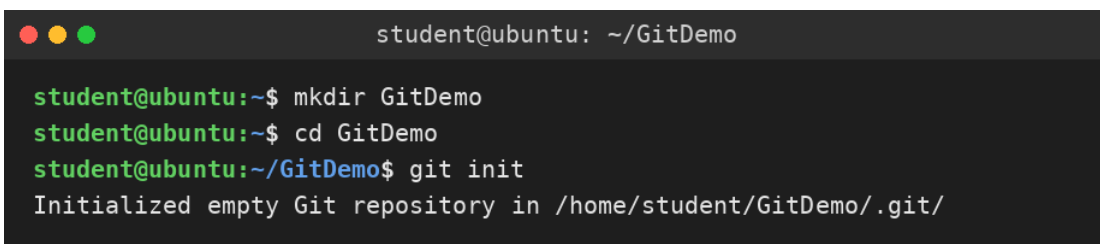
```
git config --global user.name "Student"
git config --global user.email "student@example.com"
```

A terminal window titled 'student@ubuntu: ~' showing two 'git config' commands being executed. The first sets the global user name to 'Student', and the second sets the global user email to 'student@example.com'. The output for the second command is '(no output; configuration saved to ~/.gitconfig)'.

```
student@ubuntu:~$ git config --global user.name "Student"
student@ubuntu:~$ git config --global user.email "student@example.com"
(no output; configuration saved to ~/.gitconfig)
```

Step 3: Initialize a New Git Repository

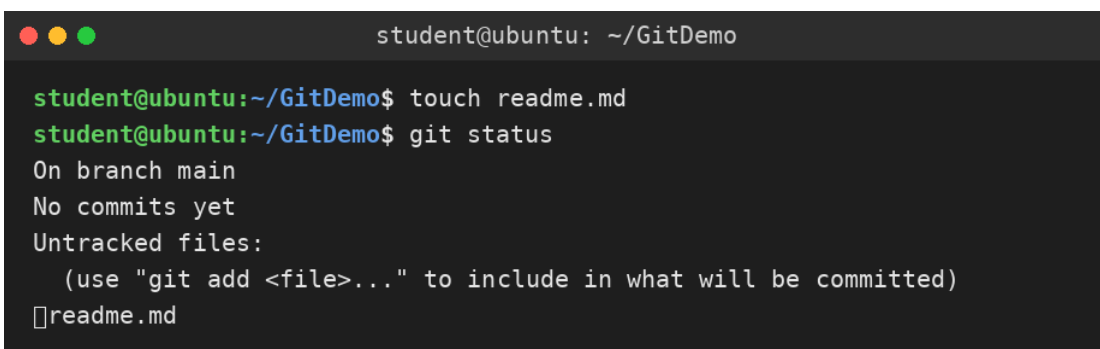
```
mkdir GitDemo
cd GitDemo
git init
```

A terminal window titled 'student@ubuntu: ~/GitDemo' showing the commands 'mkdir GitDemo', 'cd GitDemo', and 'git init' being executed. The output for 'git init' is 'Initialized empty Git repository in /home/student/GitDemo/.git/'.

```
student@ubuntu:~$ mkdir GitDemo
student@ubuntu:~$ cd GitDemo
student@ubuntu:~/GitDemo$ git init
Initialized empty Git repository in /home/student/GitDemo/.git/
```

Step 4: Check Repository Status

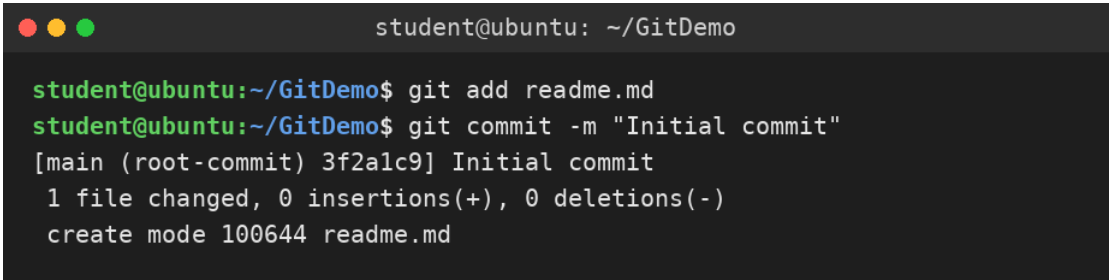
```
touch readme.md
git status
```

A terminal window titled 'student@ubuntu: ~/GitDemo' showing the commands 'touch readme.md' and 'git status' being executed. The output for 'git status' shows the current branch is 'main', there are no commits yet, and 'readme.md' is an untracked file.

```
student@ubuntu:~/GitDemo$ touch readme.md
student@ubuntu:~/GitDemo$ git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  readme.md
```

Step 5: Stage and Commit Changes

```
git add readme.md
git commit -m "Initial commit"
```

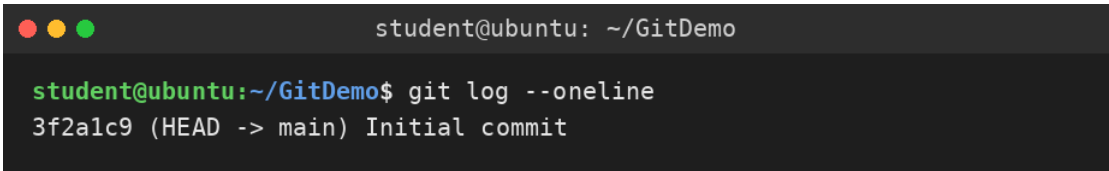


```
student@ubuntu: ~/GitDemo

student@ubuntu:~/GitDemo$ git add readme.md
student@ubuntu:~/GitDemo$ git commit -m "Initial commit"
[main (root-commit) 3f2a1c9] Initial commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 readme.md
```

Step 18: View Commit History

```
git log --oneline
```

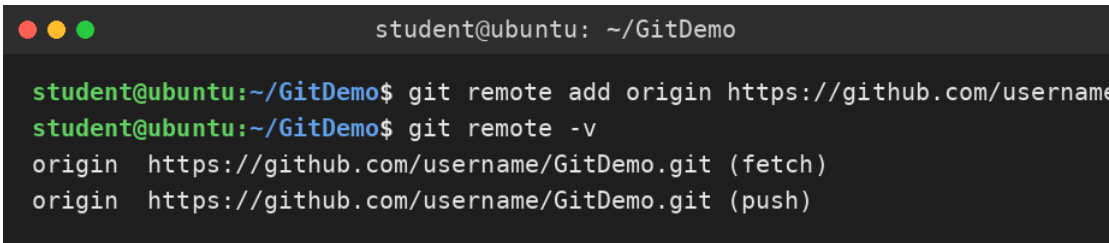


```
student@ubuntu: ~/GitDemo

student@ubuntu:~/GitDemo$ git log --oneline
3f2a1c9 (HEAD -> main) Initial commit
```

Step 6: Connect to a GitHub Repository

```
git remote add origin https://github.com/username/GitDemo.git
git remote -v
```

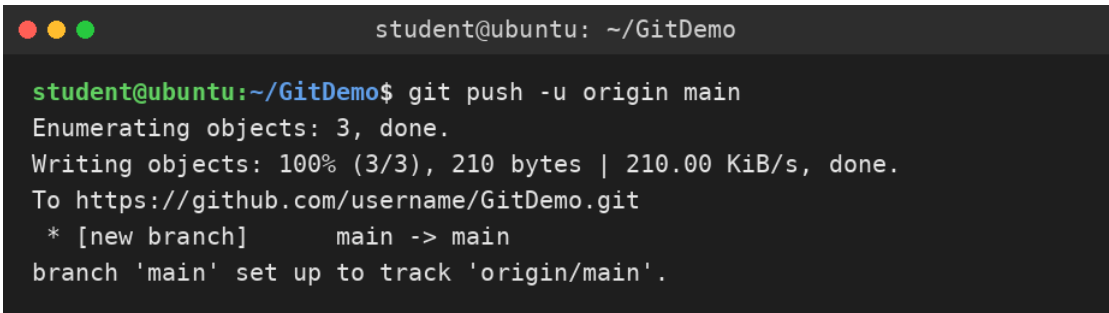


```
student@ubuntu: ~/GitDemo

student@ubuntu:~/GitDemo$ git remote add origin https://github.com/username/GitDemo.git
student@ubuntu:~/GitDemo$ git remote -v
origin  https://github.com/username/GitDemo.git (fetch)
origin  https://github.com/username/GitDemo.git (push)
```

Step 7: Push Changes to GitHub

```
git push -u origin main
```

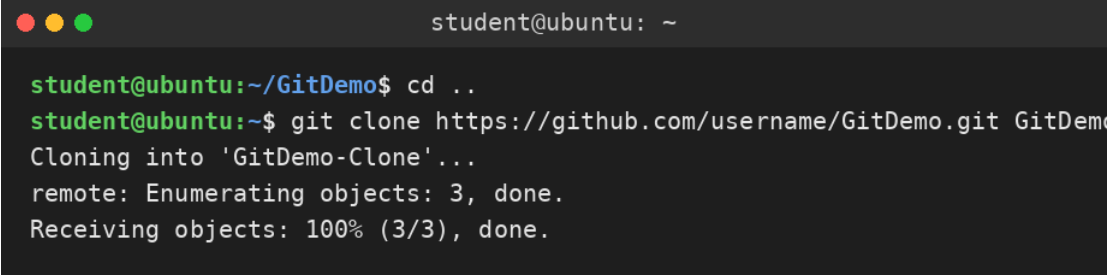


```
student@ubuntu: ~/GitDemo

student@ubuntu:~/GitDemo$ git push -u origin main
Enumerating objects: 3, done.
Writing objects: 100% (3/3), 210 bytes | 210.00 KiB/s, done.
To https://github.com/username/GitDemo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Step 8: Clone an Existing Repository

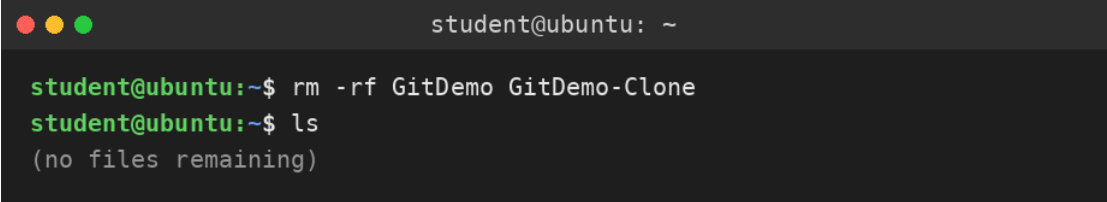
```
cd ..  
git clone https://github.com/username/GitDemo.git GitDemo-Clone
```

A terminal window titled 'student@ubuntu: ~' showing the execution of the 'git clone' command. The user is in the directory ~/GitDemo and runs 'cd ..'. Then, they run 'git clone https://github.com/username/GitDemo.git GitDemo-Clone'. The output shows the cloning process: 'Cloning into 'GitDemo-Clone'...', 'remote: Enumerating objects: 3, done.', and 'Receiving objects: 100% (3/3), done.'.

```
student@ubuntu: ~  
  
student@ubuntu:~/GitDemo$ cd ..  
student@ubuntu:~$ git clone https://github.com/username/GitDemo.git GitDemo-Clone  
Cloning into 'GitDemo-Clone'...  
remote: Enumerating objects: 3, done.  
Receiving objects: 100% (3/3), done.
```

Step 9: Clean Up Git Demo Folders

```
rm -rf GitDemo GitDemo-Clone  
ls
```

A terminal window titled 'student@ubuntu: ~' showing the cleanup process. The user runs 'rm -rf GitDemo GitDemo-Clone' and then 'ls'. The output of 'ls' is '(no files remaining)'.

```
student@ubuntu: ~  
  
student@ubuntu:~$ rm -rf GitDemo GitDemo-Clone  
student@ubuntu:~$ ls  
(no files remaining)
```

Command Reference

Command	Description
<code>pwd</code>	Prints the current working directory.
<code>ls</code>	Lists files and directories in the current location.
<code>ls -l</code>	Lists files in long format (permissions, owner, size, date).
<code>mkdir</code>	Creates a new directory.
<code>cd</code>	Changes the current directory.
<code>cd ..</code>	Moves up one directory level to the parent.
<code>touch</code>	Creates a new empty file (or updates its timestamp).
<code>nano</code>	Opens a simple terminal-based text editor.
<code>cat</code>	Displays the contents of a file.
<code>cp</code>	Copies a file to a new location or name.
<code>mv</code>	Moves or renames a file.
<code>rm</code>	Deletes a file.
<code>rmdir</code>	Removes an empty directory.
<code>echo "text" > file</code>	Writes (overwrites) text into a file.
<code>echo "text" >> file</code>	Appends text to the end of a file.
<code>grep</code>	Searches for a pattern of text inside a file.
<code>chmod</code>	Changes file permissions (e.g. read/write/execute).
<code>history</code>	Displays previously run commands.
<code>rm -r</code>	Recursively deletes a directory and its contents.
<code>git init</code>	Initializes a new local Git repository.
<code>git status</code>	Shows the state of the working directory and staging area.
<code>git add</code>	Stages changes to be included in the next commit.
<code>git commit -m</code>	Records staged changes with a message.
<code>git log</code>	Shows the commit history.
<code>git remote add</code>	Links a local repository to a remote (e.g. GitHub).
<code>git push</code>	Uploads local commits to the remote repository.
<code>git clone</code>	Downloads a copy of a remote repository.

Common Errors & Troubleshooting

Troubleshooting Tips

- **No such file or directory** — The file or directory name was typed incorrectly, or you are not in the expected location. Verify with `pwd` and `ls`.
- **rmdir: failed to remove '...': Directory not empty** — `rmdir` only removes empty directories. Delete the files inside first, or use `rm -r` with caution.
- **Permission denied** — You may not have the required access rights. Check file permissions using `ls -l`, or use `sudo` only when appropriate.
- **command not found** — The command was mistyped, or the required package is not installed.
- **Accidentally exited nano without saving** — Simply repeat the step; no changes are made until `Ctrl + O` (write out) is confirmed.

Exercises

1. Create Your Personal Workspace

Create a directory named `StudentLab`. Inside it, create the following subdirectories:

- `Programs`
- `Assignments`
- `Notes`

Verify the directory structure and display the current working directory.

2. File Management Practice

Inside the `Assignments` directory:

- Create the files:
 - `assignment1.txt`
 - `assignment2.txt`
 - `assignment3.txt`
- Do all using Command Lines learned in this Lab.
- Rename `assignment3.txt` to `lab3.txt`.
- Delete `assignment2.txt`.
- Display the remaining files.

3. Creating and Editing Files

Create a file named `profile.txt` using the Nano editor.

Enter the following information:

- Name
- Enrollment Number
- Department
- Favourite Programming Language

Save the file and display its contents using the terminal.

4. File Copy and Backup

Create a file named `report.txt`.

Copy it to a new file named `report_backup.txt`.

Edit only the backup file and add one additional line.

Finally, display both files and verify that the original file remains unchanged.

5. Directory Tree Creation

Create the following directory structure:

`LinuxLab`

`C`

`C++`

`Python`

`Java`

Inside each directory, create a file named `program1.txt`.

Display the contents of every directory.

6. Directory Management

Create the following directory structure:

Projects

AI

ML

DL

Perform the following operations:

- Delete the ML directory.
- Rename the DL directory to `DeepLearning`.
- Verify the final directory structure.

7. Compile and Execute a C++ Program

Write a C++ program that accepts one string and displays is it a palindrome or not.

Compile the program using the GNU C++ compiler and execute it from the terminal.

Record both the compilation command and the program output.

8. Git Repository Basics

Create a directory named `GitPractice`.

Inside it:

- Initialize a Git repository.
- Create a file named `readme.md`.
- Stage the file.
- Commit the changes with the message `First Commit`.
- Display the repository status.
- Display the commit history.

9. Mini Linux Project

Create the following directory structure:

Library

Books

Notes

Assignments

Perform the following tasks:

- Create the files:
 - `booklist.txt`
 - `notes.txt`
 - `assignment.txt`
- Copy `booklist.txt` into the `Notes` directory.
- Rename the copied file to `library_books.txt`.
- Delete `assignment.txt`.
- Display the final directory structure.

10. Linux Command Challenge

Create the following final directory structure without using any graphical interface:

Student

Programs

`cpp.cpp`

`python.py`

Documents

`resume.txt`

Backup

`resume_backup.txt`

Requirements:

- Create all required directories.
 - Create all required files.
 - Add suitable text to `resume.txt`.
 - Copy the file into the **Backup** directory.
 - Rename the copied file to `resume_backup.txt`.
 - Remove any unnecessary files created during the process.
 - Display the final directory structure.
 - Print the current working directory.
-